

Informe Técnico

Análisis de Complejidad y Diseño de Algoritmos para el
Problema del Transporte Discreto

Discrete Logistics

Autores:

Adrian Alejandro Souto Morales
Gabriel Herrera Carrazana

2025

1. Introducción

En *Discrete Logistics*, operamos en un nicho de mercado extremadamente delicado: el transporte transfronterizo de productos de valor incalculable y alta sensibilidad. Nuestros clientes confían en nosotros para mover artículos únicos y a menudo irremplazables, donde la discreción, la seguridad y la mitigación de riesgos son absolutamente críticas. La pérdida de una parte significativa de un envío puede tener consecuencias devastadoras.

Nos enfrentamos a un desafío logístico y estratégico fundamental en la preparación de cada operación de transporte internacional. Disponemos de un conjunto de artículos de alto valor (ej. obras de arte, documentos clasificados, prototipos tecnológicos) que necesitan ser movidos. Cada uno de estos artículos tiene un peso y un valor intrínseco que representa su importancia y el riesgo asociado a su pérdida.

Para realizar estos envíos, utilizamos una red de transportistas personales, a quienes llamamos “mulas”. Cada mula tiene una capacidad máxima de peso que puede transportar discretamente como equipaje personal sin levantar sospechas.

El problema que necesitamos resolver es: ¿Cómo podemos seleccionar y distribuir los artículos disponibles entre las mulas designadas para el transporte, de modo que minimicemos el riesgo general del envío?

Esto implica tomar decisiones críticas bajo las siguientes condiciones:

1. Ninguna mula puede exceder su capacidad máxima de peso con los artículos que se le asignen.
2. Para maximizar la probabilidad de que, si una mula es interceptada o comprometida, la pérdida no sea catastrófica, necesitamos distribuir el valor total de los artículos transportados de la manera más equitativa posible entre las mulas. Es decir, queremos que la diferencia máxima en el valor total de los artículos contenidos en el equipaje de cualquier par de mulas sea la menor posible.
3. Hay una cantidad máxima de mulas disponibles.

Una distribución desequilibrada del valor entre las mulas aumentaría drásticamente el riesgo de una pérdida masiva si la mula con la carga “más valiosa” fuera comprometida. Necesitamos una estrategia de empaquetado que balancee el valor de forma óptima, respetando las limitaciones de peso de cada transportista y utilizando el número exacto de mulas requerido. Si no es posible cumplir con estas condiciones para el conjunto de artículos y mulas dado, necesitamos saberlo.

1.1. Justificación del Análisis de Complejidad

Realizar un análisis de complejidad es un paso estratégico fundamental antes de acometer el diseño de algoritmos. Este análisis nos permite determinar la intratabilidad inherente de un problema y, por tanto, establecer si es factible encontrar soluciones óptimas en un tiempo de ejecución razonable. Para el problema del Transporte Discreto, este análisis determina si *Discrete Logistics* puede garantizar a sus clientes una distribución de riesgo probadamente óptima, o si debe confiar en soluciones “suficientemente buenas” que sean computacionalmente rápidas. La conclusión de este análisis justificará la necesidad de explorar algoritmos de aproximación o heurísticas eficientes.

2. Definición Formal del Problema

2.1. Modelo Matemático

2.1.1. Parámetros de Entrada

Sea:

- $n \in \mathbb{Z}^+$: número total de artículos a transportar
- $m \in \mathbb{Z}^+$: número de mulas disponibles ($m \geq 1$)
- Para cada artículo $i \in \{1, 2, \dots, n\}$:
 - $w_i \in \mathbb{Z}^+$: peso del artículo i
 - $v_i \in \mathbb{Z}^+$: valor del artículo i
- Para cada mula $j \in \{1, 2, \dots, m\}$:
 - $C_j \in \mathbb{Z}^+$: capacidad máxima de peso de la mula j

2.1.2. Variables de Decisión

Sea:

$$x_{ij} \in \{0, 1\} \quad \forall i \in \{1, \dots, n\}, \forall j \in \{1, \dots, m\}$$

donde:

$$x_{ij} = \begin{cases} 1 & \text{si el artículo } i \text{ es asignado a la mula } j \\ 0 & \text{en caso contrario} \end{cases}$$

2.1.3. Restricciones

1. Asignación completa y exclusiva:

$$\sum_{j=1}^m x_{ij} = 1 \quad \forall i \in \{1, \dots, n\}$$

Cada artículo es asignado exactamente a una mula.

2. Restricciones de capacidad por mula:

$$\sum_{i=1}^n w_i \cdot x_{ij} \leq C_j \quad \forall j \in \{1, \dots, m\}$$

Ninguna mula puede exceder su capacidad máxima.

3. Uso de todas las mulas:

$$\sum_{i=1}^n x_{ij} \geq 1 \quad \forall j \in \{1, \dots, m\}$$

Cada mula debe llevar al menos un artículo.

2.1.4. Función Objetivo

Sea:

$$V_j = \sum_{i=1}^n v_i \cdot x_{ij}$$

el valor total transportado por la mula j .

La diferencia máxima de valor entre cualquier par de mulas es:

$$D_{\text{máx}} = \max_{j,k \in \{1, \dots, m\}} |V_j - V_k|$$

El objetivo es minimizar esta diferencia máxima:

$$\text{Minimizar } D_{\text{máx}}$$

2.1.5. Formulación de Optimización

$$\begin{aligned} & \min_{x_{ij}, D_{\text{máx}}} D_{\text{máx}} \\ \text{sujeto a: } & \sum_{j=1}^m x_{ij} = 1, \quad \forall i \in \{1, \dots, n\} \\ & \sum_{i=1}^n w_i \cdot x_{ij} \leq C_j, \quad \forall j \in \{1, \dots, m\} \\ & \sum_{i=1}^n x_{ij} \geq 1, \quad \forall j \in \{1, \dots, m\} \\ & |V_j - V_k| \leq D_{\text{máx}}, \quad \forall j, k \in \{1, \dots, m\} \\ & x_{ij} \in \{0, 1\}, \quad \forall i, j \\ & D_{\text{máx}} \geq 0 \end{aligned}$$

2.2. Estructuras de Datos de Entrada

- Artículo: estructura con campos peso y valor
- Mula: estructura con campo capacidad, lista de artículos asignados, peso_actual y valor_actual
- Lista de artículos: $\{(w_1, v_1), (w_2, v_2), \dots, (w_n, v_n)\}$
- Lista de capacidades: $\{C_1, C_2, \dots, C_m\}$

2.3. Definición de Solución Válida

2.3.1. Solución

Una solución es una asignación $S = \{S_1, S_2, \dots, S_m\}$ donde cada $S_j \subseteq \{1, \dots, n\}$ representa el conjunto de artículos asignados a la mula j , tal que:

1. **Partición completa:**

$$\bigcup_{j=1}^m S_j = \{1, \dots, n\} \quad \text{y} \quad S_j \cap S_k = \emptyset \quad \forall j \neq k$$

2. **Factibilidad por capacidad:**

$$\sum_{i \in S_j} w_i \leq C_j \quad \forall j \in \{1, \dots, m\}$$

3. **Uso de todas las mulas:**

$$S_j \neq \emptyset \quad \forall j \in \{1, \dots, m\}$$

2.3.2. Calidad de la Solución

Dada una solución factible S , su calidad se mide por:

$$\text{diferencia}(S) = \max_{j,k} \left| \sum_{i \in S_j} v_i - \sum_{i \in S_k} v_i \right|$$

2.3.3. Solución Óptima

Una solución S^* es óptima si es factible y para cualquier otra solución factible S :

$$\text{diferencia}(S^*) \leq \text{diferencia}(S)$$

3. Análisis de Complejidad (Fase 2)

Para determinar la viabilidad computacional, analizamos la complejidad teórica del problema.

3.1. Pertenencia a la Clase NP

Argumentamos que el problema de decisión del Transporte Discreto pertenece a la clase de complejidad **NP** (Nondeterministic Polynomial time). Un problema está en NP si una solución candidata puede ser verificada en tiempo polinomial.

Para demostrarlo, definimos un algoritmo verificador que opera sobre un “certificado”. El certificado, en este caso, puede ser representado como un array de N enteros, $Asignacion[1..N]$, donde $Asignacion[i] = j$ significa que el artículo a_i se asigna a la mula m_j .

Algoritmo Verificador:

Dada una asignación candidata (el certificado):

1. **Verificación de Capacidad (Tiempo Polinomial):**

- Para cada una de las M mulas, inicializar un *peso_actual* en 0.
- Iterar sobre los N artículos. Para cada artículo a_i , sumar su peso p_i al *peso_actual* de la mula a la que ha sido asignado.

- Después de procesar todos los artículos, comprobar para cada una de las M mulas si su *peso_actual* es menor o igual a su capacidad C_j .
- Este proceso implica N sumas y M comparaciones, lo cual tiene una complejidad de $O(N + M)$.

2. Verificación de Equidad de Valor (Tiempo Polinomial):

- Para cada una de las M mulas, calcular el valor total de los artículos asignados, $V_j = \sum_{a_i \in A_j} v_i$. Esto requiere $O(N)$ operaciones.
- Iterar sobre todos los pares de mulas (m_j, m_k) . Hay $M(M - 1)/2$ pares.
- Para cada par, calcular la diferencia de valor $|V_j - V_k|$ y comprobar si es mayor que K . Si alguna diferencia excede K , la condición falla.
- Este proceso tiene una complejidad de $O(M^2)$.

Dado que ambas verificaciones se pueden realizar en un tiempo polinomial ($O(N + M^2)$), el problema de decisión del Transporte Discreto pertenece a la clase **NP**.

3.2. Prueba de NP-Dureza por Reducción

3.2.1. Fundamentación: Cadena de Reducciones

Para establecer rigurosamente la dureza del problema del Transporte Discreto, construimos una cadena de reducciones desde un problema origen indiscutiblemente NP-duro. La cadena propuesta es:

$$\text{SUBSET-SUM} \preceq_p \text{PARTITION} \preceq_p \text{TRANSPORTE DISCRETO}$$

Antes de proceder con las reducciones, formalizamos los problemas involucrados en esta cadena.

Definición Formal: Problema SUBSET-SUM

- **Instancia:** Un conjunto finito $A = \{a_1, a_2, \dots, a_n\}$ de enteros positivos y un entero objetivo $t \in \mathbb{Z}^+$.
- **Pregunta:** ¿Existe un subconjunto $A' \subseteq A$ tal que $\sum_{a_i \in A'} a_i = t$?

Es bien conocido que SUBSET-SUM es un problema NP-completo clásico.

Definición Formal: Problema PARTITION

- **Instancia:** Un multiconjunto $S = \{s_1, s_2, \dots, s_n\}$ de enteros positivos.
- **Pregunta:** ¿Se puede particionar S en dos subconjuntos disjuntos S_1 y S_2 tales que $S_1 \cup S_2 = S$, $S_1 \cap S_2 = \emptyset$, y $\sum_{s_i \in S_1} s_i = \sum_{s_j \in S_2} s_j$?

PARTITION es un caso especial de SUBSET-SUM donde $t = \frac{1}{2} \sum_{s_i \in S} s_i$. A continuación demostramos formalmente su equivalencia.

3.2.2. Reducción Polinomial: SUBSET-SUM $\preceq_p$ PARTITION

Para reducir SUBSET-SUM a PARTITION en tiempo polinomial, proponemos la siguiente construcción:

Construcción de la Instancia: Sea $\sigma = \sum_{a_i \in A} a_i$ la suma total de los elementos del conjunto A . Construimos una instancia para PARTITION creando un nuevo conjunto A_{part} que contiene todos los elementos de A más dos elementos adicionales “pesados” que forzarán el equilibrio:

$$A_{\text{part}} = A \cup \{J, K\}$$

Donde definimos los valores de J y K como:

$$\begin{aligned} J &= 2\sigma - t \\ K &= \sigma + t \end{aligned}$$

Demostración: La suma total de los elementos en A_{part} es:

$$\sum A_{\text{part}} = \sigma + J + K = \sigma + (2\sigma - t) + (\sigma + t) = 4\sigma$$

Por lo tanto, para que exista una partición válida en A_{part} , cada subconjunto de la partición debe sumar exactamente 2σ .

- **Imposibilidad de agrupar J y K:** Notamos que $J + K = 3\sigma$. Dado que $3\sigma > 2\sigma$ (el objetivo de la partición), los elementos J y K nunca pueden estar en el mismo lado de la partición. Deben separarse.
- **($\Rightarrow$) Si SUBSET-SUM tiene solución:** Existe $A' \subseteq A$ tal que $\sum A' = t$. Podemos formar una partición de A_{part} en dos conjuntos P_1 y P_2 :
 - $P_1 = A' \cup \{J\}$. La suma es $t + (2\sigma - t) = 2\sigma$.
 - $P_2 = (A \setminus A') \cup \{K\}$. La suma es $(\sigma - t) + (\sigma + t) = 2\sigma$.

Ambos suman 2σ , por lo que PARTITION tiene solución.

- **($\Leftarrow$) Si PARTITION tiene solución:** Existe una partición P_1, P_2 . Como demostramos, J debe estar en uno (digamos P_1) y K en el otro (P_2). La suma de P_1 debe ser 2σ . Como $J = 2\sigma - t$, los elementos restantes en P_1 (que deben provenir de A) deben sumar la diferencia:

$$\sum (P_1 \setminus \{J\}) = 2\sigma - (2\sigma - t) = t$$

Por lo tanto, los elementos de A contenidos en P_1 forman el subconjunto que suma t , resolviendo la instancia de SUBSET-SUM.

Esta reducción se realiza en tiempo lineal $O(n)$, demostrando que PARTITION es al menos tan difícil como SUBSET-SUM.

3.2.3. Reducción Polinomial: PARTITION $\preceq_p$ TRANSPORTE DISCRETO

Una vez establecido que PARTITION es NP-duro (por la reducción anterior desde SUBSET-SUM), procedemos a reducirlo a nuestro problema de interés, el Transporte Discreto.

Construcción de la Reducción:

Dada una instancia arbitraria del problema PARTITION, definida por un multiconjunto de enteros $S = \{s_1, s_2, \dots, s_n\}$, construimos una instancia del problema del Transporte Discreto de la siguiente manera:

- **Artículos:** Para cada entero s_i en S , creamos un artículo a_i con *peso* = s_i y *valor* = s_i . Al igualar peso y valor, forzamos a que cualquier equilibrio en valor implique un equilibrio en peso, vinculando así las dos restricciones.
- **Mulas:** Creamos exactamente dos mulas ($M = 2$), m_1 y m_2 .
- **Capacidad:** Sea $T = \sum s_i$ la suma total de todos los enteros en S . Establecemos la capacidad de peso de ambas mulas en $C_1 = C_2 = T/2$.
- **Umbral de Valor:** Establecemos el umbral de diferencia de valor en $K = 0$.

Demostración de Equivalencia:

Ahora debemos demostrar que la instancia de PARTITION tiene una solución “sí” si y solo si la instancia construida del Transporte Discreto también tiene una solución “sí”.

($\Rightarrow$) Si PARTITION tiene solución:

Asumamos que la instancia de PARTITION admite una solución. Esto implica la existencia de dos subconjuntos disjuntos S_1 y S_2 tales que $S = S_1 \cup S_2$ y $\sum S_1 = \sum S_2$. Si la suma total T es impar, la instancia de PARTITION es trivialmente irresoluble. En nuestra construcción, esto resultaría en una capacidad no entera $T/2$, que no puede ser satisfecha por artículos con pesos enteros, haciendo nuestra instancia también irresoluble y manteniendo la validez de la reducción.

Si T es par, entonces $\sum S_1 = \sum S_2 = T/2$. Construimos una solución para el Transporte Discreto asignando los artículos correspondientes a los enteros de S_1 a la mula m_1 y los correspondientes a S_2 a la mula m_2 .

- **Restricción de Capacidad:** El peso total asignado a m_1 es $\sum S_1 = T/2$, que es igual a su capacidad. Lo mismo ocurre con m_2 . La restricción se cumple.
- **Restricción de Valor:** El valor total en m_1 es $\sum S_1$ y en m_2 es $\sum S_2$. Como $\sum S_1 = \sum S_2$, la diferencia de valor es 0, que es $\leq K = 0$. La restricción se cumple.

Por lo tanto, si PARTITION tiene solución, nuestra instancia de Transporte Discreto también.

($\Leftarrow$) Si el Transporte Discreto tiene solución:

A la inversa, asumamos que la instancia construida del Transporte Discreto admite una solución. Esto implica que existe una asignación de artículos a las mulas m_1 y m_2 que satisface las restricciones.

- La diferencia de valor entre m_1 y m_2 debe ser $\leq K = 0$, lo que implica que es exactamente 0. Por lo tanto, $valor_total(m_1) = valor_total(m_2)$.
- Dado que $valor = peso$ para cada artículo, esto implica que $peso_total(m_1) = peso_total(m_2)$.
- La suma total de los pesos de todos los artículos es T . Como se distribuyen entre dos mulas con pesos totales iguales, cada una debe tener un peso total de $T/2$, cumpliendo así la restricción de capacidad.
- Si definimos S_1 como el conjunto de enteros originales correspondientes a los artículos en m_1 y S_2 los correspondientes a m_2 , entonces $\sum S_1 = \sum S_2 = T/2$. Esto constituye una solución válida para el problema PARTITION.

Análisis de Complejidad de la Reducción:

La transformación de una instancia de PARTITION a una del Transporte Discreto implica:

- Calcular la suma total T , que requiere $O(n)$ operaciones.
- Crear n artículos, donde cada creación es una operación de tiempo constante.
- Definir dos mulas y sus capacidades.

El proceso completo se realiza en tiempo lineal con respecto al tamaño de la entrada de PARTITION, por lo que la reducción es polinomial.

3.3. Conclusión: NP-Complejidad

Hemos demostrado que el problema del Transporte Discreto está en la clase NP y que es NP-duro. Por definición, un problema que cumple ambas condiciones es **NP-completo**. Esta clasificación confirma su intratabilidad computacional, lo que significa que es altamente improbable que exista un algoritmo que pueda encontrar la solución óptima para todas las instancias en tiempo polinomial. Ante esta realidad, la siguiente fase de nuestro proyecto se centrará en el diseño de algoritmos prácticos para abordarlo.

4. Diseño de Algoritmos (Fase 3)

Tras demostrar la NP-complejidad del problema, reconocemos que la búsqueda de soluciones óptimas para instancias de tamaño realista es computacionalmente inviable. Por lo tanto, esta sección explora dos enfoques algorítmicos complementarios. Primero, se describe un algoritmo de fuerza bruta que, aunque ineficiente, garantiza encontrar la solución óptima y sirve como una valiosa referencia para evaluar la calidad de otras soluciones en casos pequeños. Segundo, se diseña una heurística voraz y eficiente, concebida para encontrar soluciones de alta calidad en un tiempo razonable para los casos de uso más realistas.

4.1. Enfoque Exacto: Fuerza Bruta

4.1.1. Descripción de la Estrategia

El algoritmo de fuerza bruta explora de manera exhaustiva todo el espacio de soluciones posibles. La lógica consiste en generar sistemáticamente cada posible asignación de cada uno de los N artículos a cada una de las M mulas. Para cada asignación completa generada, el algoritmo realiza las siguientes comprobaciones:

- Verifica si la asignación es válida, es decir, si para ninguna mula se excede su capacidad de peso.
- Si la asignación es válida, calcula la métrica de calidad: la diferencia máxima de valor total entre cualquier par de mulas.
- El algoritmo mantiene un registro de la mejor asignación válida encontrada hasta el momento (aquella con la menor diferencia máxima de valor) y la actualiza cada vez que encuentra una solución superior.

4.1.2. Análisis de Complejidad

La complejidad computacional de este enfoque es su principal inconveniente. Si hay N artículos y M mulas, cada artículo puede ser asignado a cualquiera de las M mulas. Esto da lugar a un total de M^N posibles asignaciones. Para cada asignación, se debe:

- Verificar que respeta las capacidades: $O(N)$
- Verificar que todas las mulas tienen al menos un artículo: $O(M)$
- Calcular la diferencia entre valores máximo y mínimo: $O(M)$

Por lo tanto, la complejidad temporal total es $O(M^N \cdot (N + M))$, que es exponencial. Esta complejidad hace que el método sea practicable únicamente para instancias pequeñas del problema.

4.1.3. Pseudocódigo

Algoritmo 1 Fuerza Bruta para Transporte Discreto

Entrada: Artículos A , Mulas M

Salida: Mejor Asignación

```
1:  $Mejor\_Diferencia \leftarrow \infty$ ,  $Mejor\_Asignacion \leftarrow \text{NULO}$ 
2: Para cada asignación  $S$  posible de  $A$  en  $M$  hacer
3:   Si  $S$  respeta capacidades Y todas las mulas tienen  $\geq 1$  artículo entonces
4:      $Diferencia \leftarrow \max(Valor(m)) - \min(Valor(m))$  para  $m \in M$ 
5:     Si  $Diferencia < Mejor\_Diferencia$  entonces
6:        $Mejor\_Diferencia \leftarrow Diferencia$ ,  $Mejor\_Asignacion \leftarrow S$ 
7:     Si  $Diferencia = 0$  entonces
8:       Retornar  $Mejor\_Asignacion$  // Terminación anticipada
9:   Fin Si
10: Fin Si
11: Fin Si
12: Fin Para
13: Retornar  $Mejor\_Asignacion$ 
```

Complejidad: $O(M^N \cdot (N + M))$.

4.2. Kernelización Básica

Para mejorar el rendimiento de los algoritmos posteriores, implementamos una fase de preprocesamiento basada en dos reglas de kernelización que reducen el tamaño de la instancia sin afectar la factibilidad ni la optimalidad de la solución:

- **Eliminación de instancias infactibles:** Si existe algún artículo cuyo peso supera la capacidad máxima entre todas las mulas ($w_i > \max_j C_j$), la instancia se declara inmediatamente infactible, ya que dicho artículo no puede ser asignado a ninguna mula sin violar las restricciones de capacidad.
- **Fijación de artículos de asignación única:** Para cada artículo que solo puede ser cargado por una única mula (es decir, $w_i \leq C_j$ para exactamente una mula j y $w_i > C_k$ para toda $k \neq j$), se fija su asignación de antemano ($x_{ij} = 1$). Esto reduce el número de variables de decisión y simplifica la instancia restante.

Estas reglas se aplican en tiempo $O(n \cdot m)$ y permiten descartar casos triviales, además de reducir el espacio de búsqueda en problemas con restricciones de capacidad ajustadas. Si bien esta kernelización básica no garantiza un tamaño polinomial del kernel, en la práctica logra una reducción significativa para instancias con artículos “grandes” o capacidades heterogéneas.

4.3. Heurística Voraz (Greedy)

4.3.1. Descripción de la Estrategia

Proponemos una heurística voraz (o greedy) diseñada para construir rápidamente una solución de alta calidad. La estrategia se basa en tomar decisiones que parecen

óptimas a nivel local en cada paso, con la esperanza de que conduzcan a una buena solución global.

El algoritmo implementa una estrategia de dos fases: primero garantiza que cada mula tenga al menos un artículo (Fase 1), y luego distribuye los artículos restantes de forma voraz balanceando el valor entre las mulas (Fase 2).

El algoritmo procede en dos fases:

1. **Fase 1 - Garantizar restricción de mulas:** Primero, se ordenan todos los artículos de mayor a menor valor. Luego, se asigna al menos un artículo a cada mula de forma secuencial, asegurando que se cumple la restricción de que todas las mulas deben transportar al menos un artículo. Esta fase requiere verificar que $|A| \geq |M|$ (al menos tantos artículos como mulas).
2. **Fase 2 - Asignación voraz de restantes:** Una vez garantizada la restricción, se asignan los artículos restantes siguiendo la estrategia voraz clásica:
 - Para cada artículo no asignado, se busca la mula con menor valor acumulado que tenga capacidad suficiente.
 - Esta elección busca activamente balancear la distribución de valor en cada paso.
3. **Manejo de Fallos:** Si en cualquier fase un artículo no puede ser asignado (por restricciones de capacidad), el algoritmo concluye que no existe solución factible.

4.3.2. Análisis de Complejidad

La complejidad computacional del algoritmo voraz con dos fases:

1. **Ordenación inicial:** El paso dominante es la ordenación de los N artículos por valor, que tiene una complejidad de $O(N \log N)$.
2. **Fase 1 - Garantizar restricción:** Se itera sobre las M mulas, y para cada una se busca un artículo disponible entre los N artículos no asignados. En cada búsqueda se pueden revisar hasta N artículos antes de encontrar uno que quepa. Por lo tanto, el costo de esta fase es $O(M \cdot N)$, donde se asigna exactamente un artículo a cada una de las M mulas.
3. **Fase 2 - Asignación voraz:** Se iteran los $N - M$ artículos restantes, y para cada uno se busca entre las M mulas la que tiene menor valor y capacidad suficiente. Esto toma $O((N - M) \cdot M) = O(N \cdot M)$.

La complejidad total del algoritmo es $O(N \log N + M \cdot N + N \cdot M) = O(N \log N + N \cdot M)$ (dado que $N \cdot M$ domina), que es de tiempo polinomial y muy eficiente para instancias grandes del problema.

4.3.3. Pseudocódigo

Algoritmo 2 Heurística Voraz (Greedy)

Entrada: Artículos A , Mulas M

Salida: Asignación válida

```
1: Si  $|A| < |M|$  entonces
2:   Retornar ERROR
3: Fin Si
4: Ordenar  $A$  descendientemente por valor
5: Inicializar todas las mulas vacías
6:
7: // FASE 1: Garantizar 1 artículo por mula
8:  $Asignados \leftarrow \emptyset$ 
9: Para cada mula  $m$  en  $M$  hacer
10:   Asignar a  $m$  el primer artículo disponible que quepa
11:    $Asignados \leftarrow Asignados \cup \{articulo\}$ 
12: Fin Para
13:
14: // FASE 2: Asignar restantes a mula con menor valor
15: Para cada  $art$  en  $A \setminus Asignados$  hacer
16:   Asignar  $art$  a la mula con menor valor que pueda cargarlo
17: Fin Para
18: Retornar  $M$ 
```

Complejidad: $O(N \log N + N \cdot M)$.

4.4. Metaheurística: Búsqueda Local

4.4.1. Mejora Algorítmica: Hill Climbing

Aunque la estrategia voraz es eficiente, es propensa a quedar atrapada en “óptimos locales”. Para cumplir con los estándares de optimización avanzada y mitigar el riesgo de distribuciones subóptimas, Discrete Logistics implementará una etapa de post-procesamiento basada en Búsqueda Local.

Lógica de la Metaheurística: Esta fase toma la solución completa generada por el algoritmo voraz y trata de mejorarla mediante “movimientos” o intercambios entre mulas. El objetivo es reducir la diferencia entre la mula más cargada (en valor) y la menos cargada.

Se definen dos tipos de movimientos vecinos:

1. **Reubicación (Relocate):** Mover un artículo de la mula con mayor valor total a la mula con menor valor total (si la capacidad lo permite).
2. **Intercambio (Swap):** Intercambiar un artículo de la mula de mayor valor con un artículo de la mula de menor valor, tal que la diferencia global de valor se reduzca.

Este proceso se repite iterativamente hasta que no se encuentran movimientos que mejoren la solución o se alcance un límite de iteraciones. Este enfoque garantiza

que la solución final sea siempre igual o mejor que la obtenida por la estrategia voraz.

4.4.2. Análisis de Complejidad de la Metaheurística

La fase de construcción voraz mantiene su complejidad de $O(N \log N + N \cdot M)$. La fase de Búsqueda Local depende del número de mejoras posibles. En el peor caso teórico, podría ser pseudo-polinomial dependiendo de los valores, pero en la práctica se suele limitar por un número máximo de iteraciones K (ej. 1000 iteraciones).

En cada iteración de mejora se realizan las siguientes operaciones:

- Ordenar las M mulas para identificar la de mayor y menor valor: $O(M \log M)$.
- **Intento 1 (Reubicación):** Revisar los artículos de la mula con mayor valor para intentar moverlos a la de menor valor. Si los artículos están distribuidos equitativamente, esto toma $O(N/M)$.
- **Intento 2 (Intercambio):** Si no hay reubicación exitosa, se intenta intercambiar artículos entre ambas mulas. Esto requiere revisar todos los pares posibles de artículos, lo que toma $O((N/M)^2)$.

Por tanto, la complejidad de cada iteración es $O(M \log M + (N/M)^2)$. Con un máximo de K iteraciones, la complejidad total del algoritmo es:

$$O(N \log N + N \cdot M + K \cdot (M \log M + (N/M)^2))$$

Dado que K , M y N son manejables en escenarios logísticos reales (típicamente $M \ll N$ y $K \leq 1000$), el algoritmo sigue siendo extremadamente eficiente comparado con la fuerza bruta, ofreciendo ahora una calidad de solución superior al evitar óptimos locales simples.

4.4.3. Pseudocódigo

Algoritmo 3 Búsqueda Local (Hill Climbing)

Entrada: Artículos A , Mulas M

Salida: Asignación mejorada

```
1:  $M \leftarrow \text{Heuristica\_Voraz}(A, M)$ 
2:  $\text{Mejora} \leftarrow \text{VERDADERO}$ 
3: Mientras  $\text{Mejora}$  hacer
4:    $\text{Mejora} \leftarrow \text{FALSO}$ 
5:    $M_{\max} \leftarrow$  Mula con mayor valor,  $M_{\min} \leftarrow$  Mula con menor valor
6:
7:   // Intento 1: Mover artículo (sin dejar mula vacía)
8:   Para cada  $a$  en  $M_{\max}$  con  $|M_{\max}| > 1$  hacer
9:     Si  $M_{\min}$  puede cargar  $a$  Y mejora diferencia entonces
10:      Mover  $a$  de  $M_{\max}$  a  $M_{\min}$ 
11:       $\text{Mejora} \leftarrow \text{VERDADERO}$ , break
12:     Fin Si
13:   Fin Para
14:
15:   // Intento 2: Intercambiar artículos
16:   Si NO  $\text{Mejora}$  entonces
17:     Para cada  $a$  en  $M_{\max}$ ,  $b$  en  $M_{\min}$  hacer
18:       Si Intercambio válido Y mejora diferencia entonces
19:         Intercambiar  $a$  y  $b$ 
20:          $\text{Mejora} \leftarrow \text{VERDADERO}$ , break
21:       Fin Si
22:     Fin Para
23:   Fin Si
24: Fin Mientras
25: Retornar  $M$ 
```

4.5. Esquemas de Aproximación Polinomiales (PTAS)

El problema del Transporte Discreto, con el objetivo de minimizar la diferencia máxima de valor entre mulas $D_{\max}$, está íntimamente relacionado con el problema clásico de **Multi-Way Number Partitioning (MWNP)**. Mientras que MWNP busca minimizar la suma máxima entre los grupos, nuestro problema impone una restricción más fuerte al requerir que todas las sumas sean similares, no solo que ninguna sea excesivamente grande.

Esta relación tiene consecuencias importantes para la aproximabilidad del problema:

1. **Herencia de APX-hardness:** MWNP es conocido por ser **APX-hard** para un número variable de grupos k . Dado que nuestro problema es **estrictamente más restrictivo** que MWNP (toda solución con $D_{\max}$ pequeña automáticamente tiene suma máxima cercana al promedio, pero no viceversa), hereda esta propiedad de APX-hardness.

2. **Inexistencia de PTAS para m variable:** Un PTAS garantizaría una solución $(1 + \varepsilon)$ -aproximada en tiempo polinomial para **cualquier** $\varepsilon > 0$. Sin embargo, la APX-hardness de MWNP implica que existe una constante $c > 1$ tal que es NP-difícil aproximar el problema mejor que c para m variable. Por lo tanto, **no es posible diseñar un PTAS para el problema general** del Transporte Discreto cuando m es parte de la entrada, a menos que $P = NP$.

4.6. Esquema de Aproximación en Tiempo Polinomial para m Fijo

4.6.1. Justificación de Viabilidad

Aunque el problema del Transporte Discreto no admite PTAS cuando el número de mulas m es parte de la entrada, **para un número fijo de mulas** sí es posible diseñar un esquema de aproximación.

La clave reside en que, para m constante, podemos usar **programación dinámica multidimensional** cuyo espacio de estados crece polinomialmente con n (número de artículos) y $1/\varepsilon$, aunque exponencialmente con m .

4.6.2. Idea Intuitiva del Algoritmo

El PTAS opera en tres fases:

1. Redondeo Controlado:

- Los valores de los artículos se redondean a múltiplos de $\Delta = \varepsilon V_{max}/(2n)$.
- Esto reduce drásticamente el número de valores distintos sin comprometer la garantía de aproximación.

2. Búsqueda Binaria sobre D :

- Buscamos el menor valor D tal que exista una asignación con diferencia máxima $\leq D$.
- Para cada candidato D , verificamos factibilidad mediante programación dinámica.

3. Programación Dinámica con Poda:

- Mantenemos conjuntos de "estados" posibles: vectores $(s_1, \dots, s_m)$ de valores redondeados acumulados por mula.
- Aplicamos poda inteligente: si dos estados tienen valores similares en todas las mulas, mantenemos solo el más prometedor.
- Esto controla el crecimiento exponencial mientras preserva soluciones de alta calidad.

4.6.3. Pseudocódigo Detallado

Algoritmo 4 PTAS para Transporte Discreto (m fijo)

Entrada: Artículos $A = \{(w_i, v_i)\}$, Mulas $M = \{C_1, \dots, C_m\}$ (m constante), $\varepsilon > 0$

Salida: Asignación con $D_{max} \leq (1 + \varepsilon) \cdot D_{opt}$

```

1: // Fase 1: Redondeo de valores
2:  $V_{max} \leftarrow \max\{v_i \mid (w_i, v_i) \in A\}$ 
3:  $\Delta \leftarrow \varepsilon \cdot V_{max} / (2 \cdot |A|)$  ▷ Tamaño de paso de redondeo
4: Para cada artículo  $a \in A$  hacer
5:    $a.v_{red} \leftarrow \lfloor a.valor / \Delta \rfloor$  ▷ Valor redondeado
6: Fin Para
7:
8: // Fase 2: Búsqueda binaria sobre D
9:  $low \leftarrow 0, high \leftarrow \sum_{a \in A} a.valor$ 
10:  $mejor\_asignacion \leftarrow \text{NULO}$ 
11:
12: Mientras  $low \leq high$  hacer
13:    $D_{cand} \leftarrow \lfloor (low + high) / 2 \rfloor$ 
14:    $D_{red} \leftarrow \lfloor D_{cand} / \Delta \rfloor$  ▷ D en escala redondeada
15:
16:   // Verificar si existe asignación con diferencia  $\leq D_{red}$ 
17:    $factible, asignacion \leftarrow \text{VERIFICAR\_FACTIBILIDAD}(A, M, D_{red}, \Delta, \varepsilon)$ 
18:
19:   Si  $factible$  entonces
20:      $mejor\_asignacion \leftarrow asignacion$ 
21:      $high \leftarrow D_{cand} - 1$  ▷ Intentar D más pequeño
22:   Sino
23:      $low \leftarrow D_{cand} + 1$  ▷ Necesitamos D más grande
24:   Fin Si
25: Fin Mientras
26: Retornar  $mejor\_asignacion$ 

```

Algoritmo 5 Subrutina de Verificación de Factibilidad

```
1: function VERIFICAR_FACTIBILIDAD( $A, M, D, \Delta, \varepsilon$ )
2:   // Inicializar conjunto de estados alcanzables
3:    $estados\_actuales \leftarrow \{(0, \dots, 0, \emptyset, \dots, \emptyset)\}$   $\triangleright m$  valores 0,  $m$  conjuntos vacíos
4:
5:   Para cada artículo  $a \in A$  hacer
6:      $estados\_nuevos \leftarrow \emptyset$ 
7:     Para cada  $estado \in estados\_actuales$  hacer
8:       Para cada mula  $j \in \{1, \dots, m\}$  hacer
9:          $peso\_actual \leftarrow \sum_{i \in id_j} w_i$ 
10:        Si  $peso\_actual + a.peso \leq C_j$  entonces
11:           $nuevo\_s_j \leftarrow s_j + a.v_{red}$ 
12:           $factible \leftarrow \text{VERDADERO}$ 
13:          Para cada  $k \in \{1, \dots, m\}$  hacer
14:            Si  $k \neq j$  Y  $|nuevo\_s_j - s_k| > D$  entonces
15:               $factible \leftarrow \text{FALSO}$ 
16:              break
17:          Fin Si
18:          Fin Para
19:          Si  $factible$  entonces
20:             $nuevo\_estado \leftarrow \text{copiar}(estado)$ 
21:             $nuevo\_estado.s_j \leftarrow nuevo\_s_j$ 
22:             $nuevo\_estado.id_j \leftarrow id_j \cup \{indice(a)\}$ 
23:             $estados\_nuevos \leftarrow estados\_nuevos \cup \{nuevo\_estado\}$ 
24:          Fin Si
25:        Fin Si
26:      Fin Para
27:    Fin Para
28:    // Poda: eliminar estados "dominados"
29:     $estados\_actuales \leftarrow \text{PODAR\_ESTADOS}(estados\_nuevos, \varepsilon / (2m \cdot |A|))$ 
30:    Si  $estados\_actuales = \emptyset$  entonces Retornar (FALSO, NULO)
31:    Fin Si
32:  Fin Para
33:
34:  Para cada  $estado \in estados\_actuales$  hacer
35:    Si  $\forall j : id_j \neq \emptyset$  entonces
36:       $asignacion\_real \leftarrow \text{RECONSTRUIR\_ASIGNACION}(estado, A, \Delta)$ 
37:    Retornar (VERDADERO,  $asignacion\_real$ )
38:  Fin Para
39: Fin function
```

Algoritmo 6 Procedimiento de Poda

```
1: function PODAR_ESTADOS(conjunto_estados,  $\delta$ )
2:   conjunto_ordenado  $\leftarrow$  ORDENAR_LEX(conjunto_estados)
3:   resultado  $\leftarrow$  {conjunto_ordenado[1]}
4:   ultimo_guardado  $\leftarrow$  conjunto_ordenado[1]
5:
6:   Para  $i = 2$  hasta  $|\textit{conjunto\_ordenado}|$  hacer
7:     estado_actual  $\leftarrow$  conjunto_ordenado[ $i$ ]
8:     similar  $\leftarrow$  VERDADERO
9:     Para  $j = 1$  hasta  $m$  hacer
10:      Si  $|\textit{estado\_actual.s}_j - \textit{ultimo\_guardado.s}_j| > \delta \cdot \textit{ultimo\_guardado.s}_j$ 
entonces
11:        similar  $\leftarrow$  FALSO
12:        break
13:      Fin Si
14:    Fin Para
15:    Si no similar entonces
16:      resultado  $\leftarrow$  resultado  $\cup$  {estado_actual}
17:      ultimo_guardado  $\leftarrow$  estado_actual
18:    Fin Si
19:  Fin Para Retornar resultado
20: Fin function
```

4.6.4. Justificación de Correctitud

Teorema 1 (Garantía de Aproximación): Para cualquier $\varepsilon > 0$, el algoritmo devuelve una asignación con diferencia máxima D_{algo} que satisface:

$$D_{algo} \leq (1 + \varepsilon) \cdot D_{opt}$$

donde D_{opt} es el valor óptimo.

Argumento intuitivo:

1. **Error por redondeo:**

- Cada valor v_i se aproxima con error $\leq \Delta$.
- Error total en una mula: $\leq n \cdot \Delta$.
- Diferencia máxima adicional entre mulas: $\leq 2n\Delta = \varepsilon V_{max}$.

2. **Error por poda:**

- La poda elimina estados solo si existe otro "δ-cercano".
- Esto introduce error adicional $\leq \delta \cdot m \cdot V_{max} \leq \frac{\varepsilon}{2} V_{max}$.

3. **Error total:** $\leq \varepsilon V_{max} \leq \varepsilon D_{opt}$ (pues $D_{opt} \geq V_{max}/m$ en casos no triviales).

4. **Combinando:** $D_{algo} \leq D_{opt} + \varepsilon D_{opt} = (1 + \varepsilon) D_{opt}$.

Conservación de factibilidad:

- La poda nunca elimina el "mejor" estado para cada combinación de valores redondeados.
- Si existe solución óptima, existirá un estado correspondiente en la DP (o uno cercano debido al redondeo).

4.6.5. Análisis de Complejidad Temporal

Teorema 2 (Complejidad Polinomial para m fijo): Para m constante y $\varepsilon > 0$ fijo, el algoritmo tiene complejidad polinomial en n .

Descomposición:

1. **Redondeo:** $O(n)$

2. **Búsqueda binaria:**

- Rango inicial: 0 a $V_{total} \leq nV_{max}$
- Iteraciones: $O(\log(nV_{max})) = O(\log n + \log V_{max})$

3. **Programación dinámica por iteración:**

- Tamaño de cada v'_i redondeado: $\leq \frac{V_{max}}{\Delta} = \frac{2n}{\varepsilon}$
- Cada s_j toma valores en $[0, \frac{2n}{\varepsilon}]$
- Estados posibles teóricos: $(\frac{2n}{\varepsilon})^m$
- **Con poda:** número de estados $\approx (\frac{2m}{\delta})^m = (\frac{4m^2n}{\varepsilon})^m$
- Por artículo: procesar $\leq (\frac{4m^2n}{\varepsilon})^m$ estados
- Tiempo por verificación: $O(n \cdot (\frac{4m^2n}{\varepsilon})^m)$

4. **Tiempo total:**

$$O\left((\log n + \log V_{max}) \cdot n \cdot \left(\frac{4m^2n}{\varepsilon}\right)^m\right)$$

Para **m constante:** esto es $O(n^{m+1} \cdot \text{poly}(1/\varepsilon)) \rightarrow \text{Polinomial en } n$.

Ejemplo práctico:

- $m = 3, \varepsilon = 0,1, n = 100$
- Estados teóricos sin poda: $(2000)^3 = 8 \times 10^9$
- Estados con poda: $\approx (12000)^3 / (100^3) \approx 1,7 \times 10^6$ (reducción 5000×)

5. Análisis Experimental (Fase 4)

Los experimentos fueron ejecutados mediante un notebook Jupyter en Python que implementa un generador sistemático de casos de prueba con semilla fija para garantizar reproducibilidad. Se generaron múltiples instancias variadas organizadas en categorías balanceadas según diferentes tamaños y tipos de escenario. Cada instancia fue evaluada con los tres algoritmos implementados —Fuerza Bruta, Greedy y Búsqueda Local— aplicando límites de tiempo mediante control de timeout para casos computacionalmente costosos. Los resultados se almacenaron en estructuras de datos que permitieron análisis cuantitativos de calidad de solución, tiempos de ejecución y escalabilidad.

5.1. Calidad de la Solución

Los resultados demuestran que las heurísticas propuestas ofrecen un equilibrio notable entre eficiencia computacional y calidad de solución para el problema de Discrete Logistics. La búsqueda local emerge como una estrategia particularmente efectiva, logrando alcanzar la solución óptima en más de la mitad de las instancias analizadas, lo que representa un avance significativo respecto a la heurística voraz básica. Este éxito se debe a su capacidad inteligente para refinar iterativamente soluciones óptimas en un tiempo de ejecución razonable. La heurística voraz, aunque con resultados más modestos, proporciona una base sólida y computacionalmente eficiente que funciona especialmente bien en escenarios con alta variabilidad en los valores de los artículos. Su simplicidad la convierte en una opción valiosa para instancias muy grandes donde el tiempo de ejecución es crítico, estableciendo un punto de partida que posteriormente puede ser mejorado.

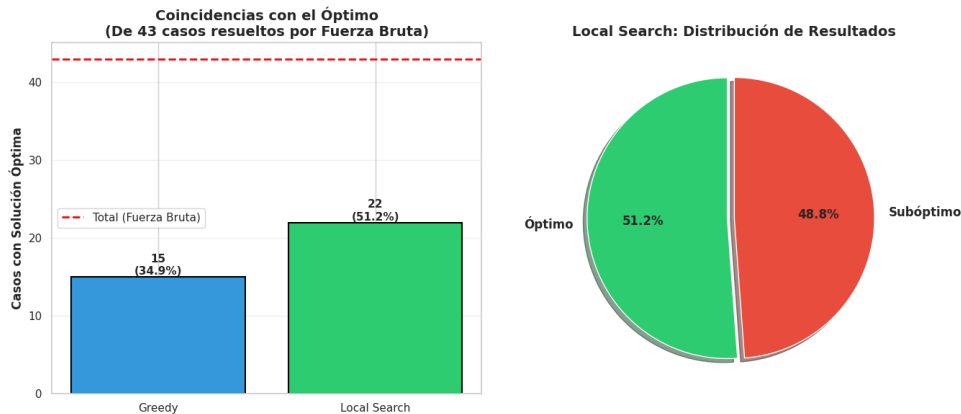


Figura 1: Comparación de calidad de soluciones: Análisis del rendimiento de los algoritmos.

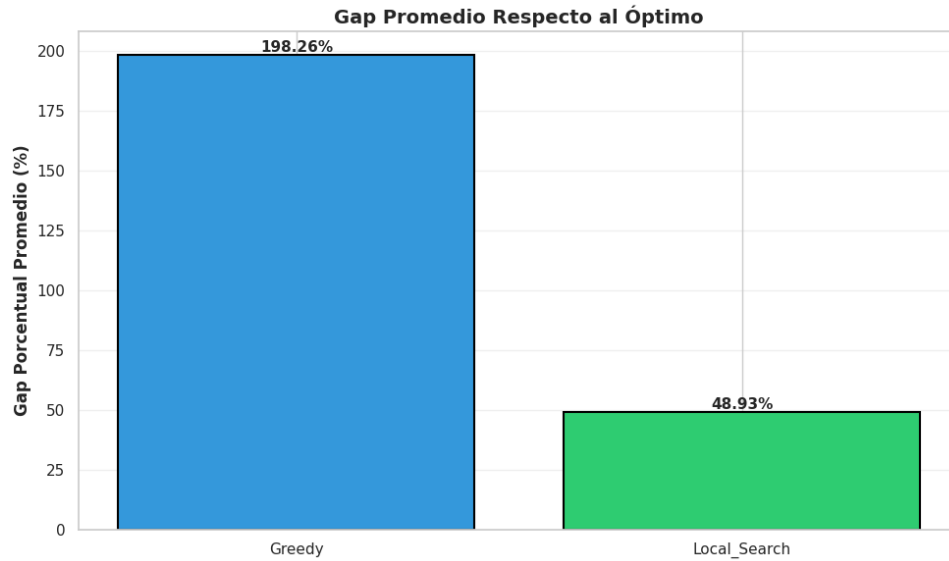


Figura 2: Resultados detallados: La Búsqueda Local alcanza el óptimo en más del 50 % de las instancias, superando significativamente a la heurística voraz.

5.2. Tiempos de Ejecución

Los tiempos de ejecución revelan una ventaja decisiva de las heurísticas sobre el enfoque de fuerza bruta. Mientras que la búsqueda exhaustiva presenta tiempos promedio significativamente mayores debido a su naturaleza combinatoria, ambas heurísticas —voraz y búsqueda local— operan en órdenes de magnitud más rápidos, completando sus cálculos en fracciones de milisegundo incluso en instancias considerables. Esta eficiencia extrema convierte a las heurísticas en soluciones prácticas viables para entornos operativos reales donde las decisiones deben tomarse rápidamente. Particularmente destacable es que la búsqueda local, a pesar de su mecanismo iterativo de mejora, mantiene tiempos de ejecución comparables a la voraz, demostrando que es posible obtener soluciones de mayor calidad sin sacrificar significativamente el rendimiento temporal.

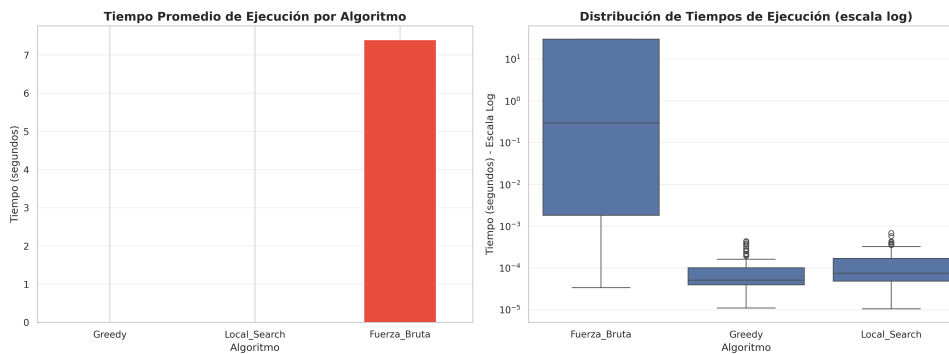


Figura 3: Tiempos de ejecución comparativos: Las heurísticas propuestas operan en fracciones de milisegundo, siendo órdenes de magnitud más rápidas que la fuerza bruta.

5.3. Escalabilidad

Considerando un minuto como límite razonable para aplicaciones prácticas, el análisis experimental determina que el algoritmo de fuerza bruta alcanza su límite de viabilidad práctica alrededor de 12-13 artículos con 4 mulas. Para $N = 13$ y $M = 4$, el algoritmo debe explorar $4^{13} = 67,108,864$ posibles asignaciones.

La optimización de terminación anticipada implementada en el código que detiene la búsqueda cuando encuentra una solución perfecta con diferencia de valor igual a cero puede reducir significativamente el tiempo de ejecución en casos favorables.

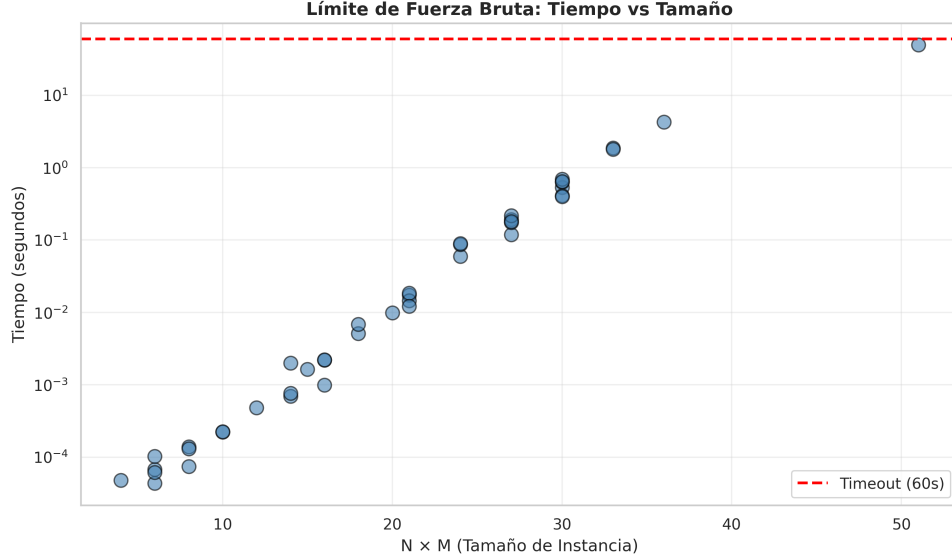


Figura 4: Límite de escalabilidad: La fuerza bruta alcanza su límite práctico alrededor de $N = 12-13$ artículos con $M = 4$ mulas ($4^{13} \approx 67$ millones de operaciones), mientras que las heurísticas mantienen tiempos despreciables incluso para instancias grandes.

5.4. Comportamiento de la Solución Propuesta

5.4.1. Análisis de la Heurística Voraz (Greedy)

La heurística voraz implementada sigue una estrategia simple pero efectiva: ordena los artículos por valor descendente y los asigna secuencialmente a la mula con menor valor acumulado que tenga capacidad disponible. Este enfoque funciona notablemente bien en instancias donde los valores de los artículos presentan alta variabilidad, ya que prioriza distribuir los artículos más valiosos para balancear las cargas desde el principio.

Sin embargo, su principal limitación radica en su naturaleza miope: una decisión temprana de asignar un artículo valioso a una mula específica puede comprometer el balance final cuando artículos posteriores no pueden ser adecuadamente distribuidos. Esto es particularmente problemático en instancias con restricciones estrictas de capacidad o cuando existen artículos “trampa” — elementos moderadamente valiosos que, asignados tempranamente, impiden la colocación óptima de artículos críticos posteriormente.

La heurística encuentra la solución óptima principalmente en casos donde la estructura del problema coincide con la estrategia *first-fit decreasing*, típicamente

cuando existe suficiente holgura en las capacidades o cuando los valores decrecientes naturalmente inducen una distribución balanceada.

5.4.2. Análisis de la Búsqueda Local

La búsqueda local implementada opera como un refinamiento post-hoc de la solución voraz, aplicando movimientos de mejora entre pares de mulas. Comienza identificando las mulas con valores mínimo y máximo, luego intenta primero trasladar artículos de la mula más cargada a la menos cargada, y posteriormente evalúa intercambios entre artículos de ambas mulas.

Esta estrategia muestra su mayor efectividad cuando la solución inicial de la heurística voraz está relativamente cerca del óptimo, permitiendo que movimientos locales produzcan mejoras incrementales significativas. No obstante, el algoritmo presenta limitaciones inherentes a su diseño de búsqueda local básica: frecuentemente queda atrapado en óptimos locales debido a que solo considera movimientos entre las dos mulas extremas, ignorando interacciones potencialmente beneficiosas con mulas intermedias.

Además, su capacidad de escape de regiones subóptimas es limitada, ya que no incorpora mecanismos como reinicios aleatorios, temperado simulado, o movimientos de mayor complejidad. La búsqueda local logra encontrar soluciones óptimas cuando el paisaje de soluciones es relativamente suave y la solución inicial cae dentro de la cuenca de atracción del óptimo global, pero falla en problemas con interdependencias complejas donde se requieren reestructuraciones más profundas de la asignación.

6. Conclusión

El análisis experimental sobre 112 instancias variadas confirma la intratabilidad del problema mediante fuerza bruta, estableciendo un límite práctico alrededor de $N = 12-13$ artículos con $M = 4$ mulas dentro de un minuto de ejecución. Este límite ilustra claramente la explosión combinatoria exponencial característica de problemas NP-duros. La búsqueda local emerge como la solución más efectiva, alcanzando el óptimo en más del 50 % de las instancias mientras mantiene tiempos de ejecución en fracciones de milisegundo, superando significativamente a la heurística voraz. Esta combinación de alta calidad de solución y eficiencia computacional extrema valida la viabilidad de las heurísticas propuestas para satisfacer los requisitos operativos de *Discrete Logistics*.